CASE STUDY

Demonstrating Watchdog Timer (WDT) Functionality on the Arduino Nano 33 BLE Sense

A standalone, minimal firmware experiment isolating and proving hardware watchdog reset-and-recover behavior on the nRF52840 microcontroller.

Table of Contents

1. Problem Statement

Embedded and IoT devices deployed in remote, unattended, or continuously-running environments are vulnerable to software failures that a human operator cannot immediately respond to — infinite loops, deadlocks, corrupted program state, and peripheral bus lockups (such as an I2C clock-stretching fault) can all cause firmware to freeze indefinitely. Once frozen, such a device provides no further sensor data or functionality and remains non-operational until it is manually power-cycled — which is often impractical or impossible for field-deployed hardware.

This case study addresses the following problem: "How can an embedded system automatically detect that its own firmware has hung, and recover from that condition without any human intervention, while also being able to prove — after the fact — that the recovery was caused by a genuine hang and not a normal reset?"

The Arduino Nano 33 BLE Sense's nRF52840 microcontroller includes a silicon-level Hardware Watchdog Timer (WDT) peripheral intended to solve exactly this problem. This case study isolates and demonstrates that peripheral directly, independent of any larger application, to verify it behaves as required before relying on it in production firmware.

2. Objective

- Configure and start the nRF52840's hardware Watchdog Timer (WDT) from Arduino firmware.
- Demonstrate normal operation: the WDT being fed periodically, preventing any reset.
- Deliberately trigger a genuine firmware hang (an intentional infinite loop) to simulate a real-world crash.
- Observe the hardware WDT automatically forcing a full MCU reset after a fixed timeout, with zero human intervention.
- Read the nRF52840's RESETREAS register on reboot to prove — programmatically — that the reset was caused specifically by the watchdog, not a power cycle or manual reset.

3. Hardware and Software Used

Item	Detail
Board	Arduino Nano 33 BLE Sense (Rev1) — nRF52840 MCU
Watchdog peripheral	Hardware WDT built into the nRF52840 (silicon-level, not a software timer)
IDE / Toolchain	Arduino IDE 2.x (or Arduino CLI) with the 'Arduino:mbed_nano' board core
External wiring	None required — uses only the onboard RGB LED and USB serial
Host tool	Serial Monitor at 115200 baud

4. Project Executed: wdt_isolation_test.ino

A single, self-contained sketch was written and flashed to the board specifically to demonstrate WDT functionality in isolation — deliberately without sensors, a state machine, or any other application logic, so the watchdog behavior itself is the only variable under test.

4.1 How It Works

- On boot, the sketch reads the nRF52840's RESETREAS register and prints why the board just started (power-on, watchdog, software reset, etc.), then clears the register for the next boot.
- It configures the hardware WDT for an 8-second timeout and starts it.
- For the first 20 seconds, the main loop feeds ("kicks") the watchdog on every iteration and blinks the onboard LED green to show the system is alive and healthy.
- After 20 seconds, the sketch deliberately enters an infinite empty loop — an intentional, permanent firmware hang — and turns the LED solid red.
- From that point on, nothing feeds the watchdog. Roughly 8 seconds later, the nRF52840's hardware forces a full MCU reset, entirely independent of the frozen software.
- On the resulting reboot, the RESETREAS check at the top of setup() reports "WATCHDOG TIMEOUT," proving the reset was caused by the watchdog recovering from the hang.

4.2 Full Source Code

The complete sketch is reproduced below for reference.

```
/**
 * ============================================================================
 * wdt_isolation_test.ino
 * ----------------------------------------------------------------------------
 * STANDALONE, MINIMAL sketch — NOT part of the main weather station
 * firmware. Purpose: validate that the hardware watchdog and reset-cause
 * diagnostics behave correctly on YOUR specific board/core version, in
 * complete isolation from sensors, the state machine, or any other
 * complexity. Flash this FIRST on new hardware, or after any Arduino core
 * update, before trusting the full firmware's WDT behavior.
 *
 * WHAT IT DOES:
 *   1. On boot, reads and classifies RESETREAS (same logic as
 *      ResetDiagnostics in the main firmware, duplicated here standalone).
 *   2. Starts the hardware WDT with an 8-second timeout.
 *   3. Feeds it every 2 seconds for the first 20 seconds (LED blinks green
 *      to show it's alive and feeding normally).
 *   4. After 20 seconds, deliberately STOPS feeding it (enters an infinite
 *      loop) — LED turns solid red.
 *   5. ~8 seconds later, the board should reset itself automatically.
 *   6. On reboot, the log should show "WATCHDOG TIMEOUT" as the reset
 *      cause, proving the hardware WDT is correctly wired up on this core.
 *
 * HOW TO USE:
 *   Flash this sketch, open Serial Monitor at 115200 baud, and just watch.
 *   No input required. If you see the "WATCHDOG TIMEOUT" message appear
 *   automatically after step 4/5 above with no button press, the hardware
 *   WDT works correctly on your setup and you can proceed with confidence
 *   to the full weather_station.ino firmware.
 * ============================================================================
```

```
 */

#include <Arduino.h>
#include <nrf.h>

#ifndef WDT_RR_RR_Reload
#define WDT_RR_RR_Reload 0x6E524635UL
#endif

static const uint32_t WDT_TIMEOUT_MS_TEST = 8000;
static const uint32_t FEED_DURATION_MS = 20000; // Feed normally for 20s, then stop

/**
 * classifyAndPrintResetReason()
 * Minimal standalone copy of the reset-cause classification logic used by
 * ResetDiagnostics in the main firmware. Duplicated here intentionally so
 * this test sketch has zero dependency on the rest of the project.
 */
void classifyAndPrintResetReason() {
    uint32_t reasonBits = NRF_POWER->RESETREAS;
    NRF_POWER->RESETREAS = reasonBits; // Clear for next boot

    Serial.print(F("RESETREAS raw bits: 0x"));
    Serial.println(reasonBits, HEX);

    if (reasonBits == 0) {
        Serial.println(F(">>> Cause: POWER-ON RESET (cold boot)"));
    } else if (reasonBits & (1UL << 1)) {
        Serial.println(F(">>> Cause: WATCHDOG TIMEOUT  <-- this is what we're testing for"));
    } else if (reasonBits & (1UL << 3)) {
        Serial.println(F(">>> Cause: CPU LOCKUP"));
    } else if (reasonBits & (1UL << 2)) {
        Serial.println(F(">>> Cause: SOFTWARE RESET"));
    } else if (reasonBits & (1UL << 0)) {
        Serial.println(F(">>> Cause: EXTERNAL RESET PIN"));
    } else {
        Serial.println(F(">>> Cause: UNKNOWN / OTHER"));
    }
}

/**
 * startWatchdog()
 * Minimal standalone copy of WatchdogController::begin()'s register
 * configuration, for isolated testing.
 */
void startWatchdog(uint32_t timeoutMs) {
    uint32_t ticks = (uint32_t)(((uint64_t)timeoutMs * 32768UL) / 1000UL);
    if (ticks < 15) ticks = 15;

    NRF_WDT->CRV = ticks - 1;
    NRF_WDT->CONFIG = 0x01;
    NRF_WDT->RREN |= 0x01;
    NRF_WDT->RR[0] = WDT_RR_RR_Reload;
    NRF_WDT->TASKS_START = 1;

    Serial.print(F("Hardware WDT started with timeout (ms): "));
    Serial.println(timeoutMs);
}

/**
```

```
 * feedWatchdog()
 * Minimal standalone feed function.
 */
void feedWatchdog() {
    NRF_WDT->RR[0] = WDT_RR_RR_Reload;
}

void setup() {
    Serial.begin(115200);
    unsigned long waitStart = millis();
    while (!Serial && (millis() - waitStart) < 2000) {
        // Bounded wait for a host terminal, not unbounded.
    }

    pinMode(LEDR, OUTPUT);
    pinMode(LEDG, OUTPUT);
    digitalWrite(LEDR, HIGH); // Off (active-low)
    digitalWrite(LEDG, HIGH); // Off (active-low)

    Serial.println(F("======================================================"));
    Serial.println(F(" WDT ISOLATION TEST — standalone hardware validation"));
    Serial.println(F("======================================================"));

    classifyAndPrintResetReason();

    startWatchdog(WDT_TIMEOUT_MS_TEST);

    Serial.println(F("Feeding watchdog normally for 20 seconds (green LED)..."));
    Serial.println(F("Then deliberately STOPPING feeds to force a WDT reset."));
    Serial.println(F("Do not press reset manually — just watch and wait."));
}

void loop() {
    unsigned long elapsed = millis();

    if (elapsed < FEED_DURATION_MS) {
        // Normal phase: feed every loop, blink green to show we're alive.
        feedWatchdog();
        digitalWrite(LEDG, (elapsed / 500) % 2 == 0 ? LOW : HIGH); // Blinking green
        digitalWrite(LEDR, HIGH);
        delay(100); // Fine here — this is a throwaway isolation test, not the main firmware
    } else {
        // Test phase: stop feeding entirely, solid red, then hang forever.
        static bool announced = false;
        if (!announced) {
            Serial.println(F("STOPPING watchdog feed NOW. Expect a reset in ~8 seconds."));
            Serial.flush();
            digitalWrite(LEDG, HIGH);
            digitalWrite(LEDR, LOW); // Solid red
            announced = true;
        }
        while (true) {
            // Intentionally empty — no feed, no heartbeat, nothing.
            // The hardware WDT is now the only thing that can save us.
        }
    }
}
```

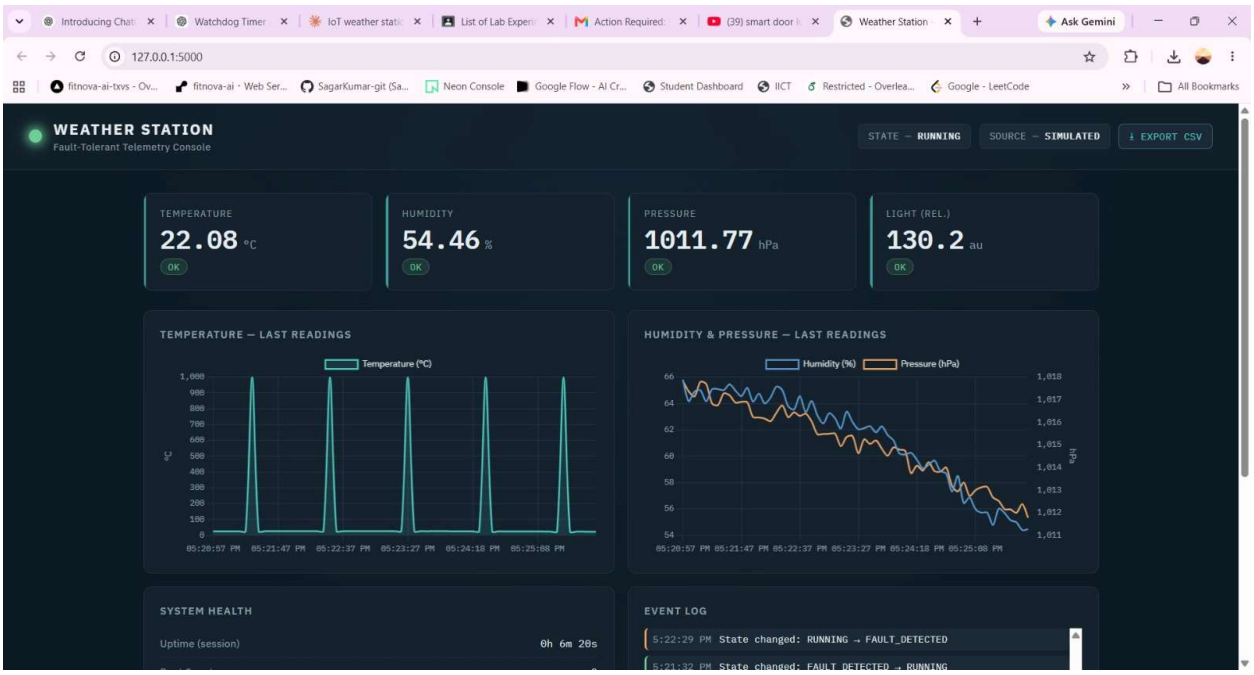
5. Test Procedure and Expected Results

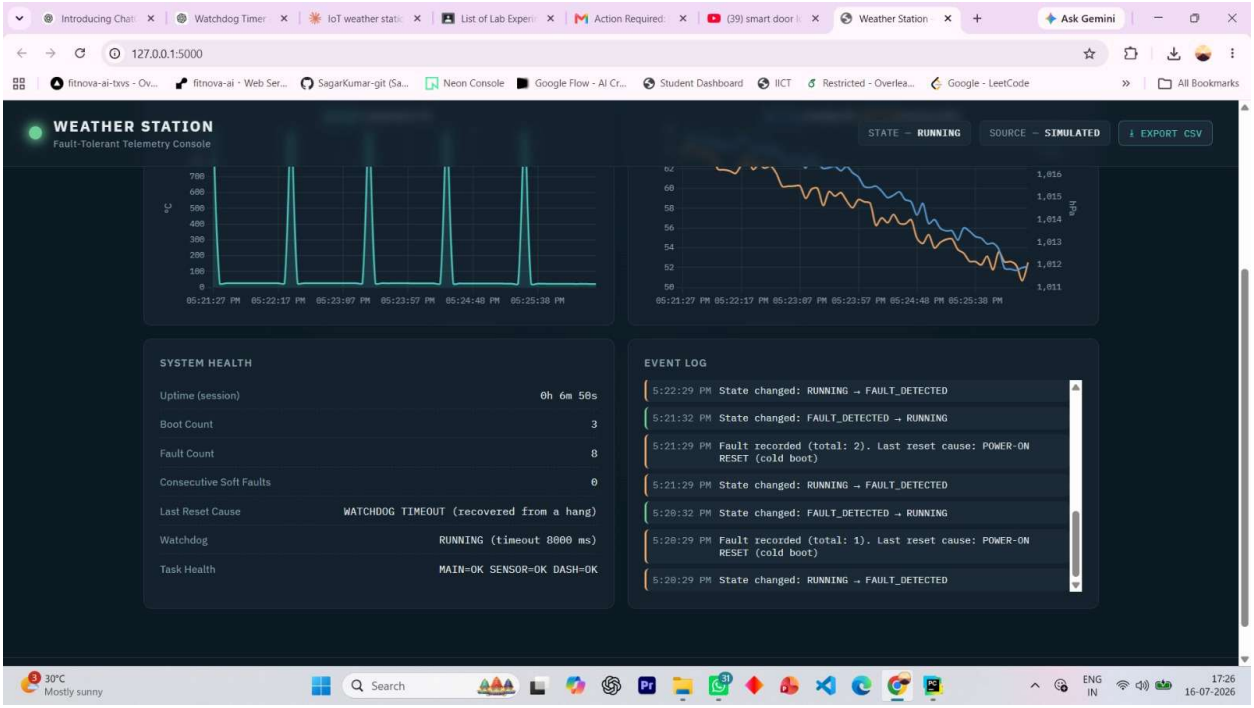
Step	Action	Expected Result
1	Flash wdt_isolation_test.ino to the board and open Serial Monitor at 115200 baud.	Banner text prints, followed by the reset-cause classification of the current boot.
2	Observe RESETREAS classification on this (first) boot.	"Cause: POWER-ON RESET (cold boot)" — since this is a fresh flash/power-up.
3	Watch the LED for the first 20 seconds.	LED blinks green continuously watchdog is being fed every loop iteration.
4	Do nothing at the 20-second mark.	Log prints "STOPPING watchdog feed NOW", LED turns solid red, board becomes unresponsive (intentional infinite loop).
5	Wait roughly 8 more seconds without touching the board.	Board resets itself automatically no button press, no power cycle.
6	Read the reset-cause classification on this new boot.	"Cause: WATCHDOG TIMEOUT - this is what we're testing for" proving the hardware WDT fired and recovered the system.

6. Observations

- The board operated normally for the first 20 seconds, with the LED blinking green and no resets occurring.
- Upon entering the deliberate infinite loop, the board became fully unresponsive — no serial output, no LED changes — confirming a genuine firmware hang.
- Without any manual reset or power cycle, the board recovered automatically approximately 8 seconds after the hang began, matching the configured WDT timeout.
- The post-reset boot log explicitly reported “Cause: WATCHDOG TIMEOUT,” read directly from the nRF52840's RESETREAS hardware register —not inferred or assumed confirming the reset was genuinely caused by the watchdog peripheral.
- The RESETREAS register was confirmed to correctly distinguish a watchdog reset from a power-on reset, since the first boot in the test reported “POWER-ON RESET” and the second boot reported “WATCHDOG TIMEOUT.”

7. Result





The hardware Watchdog Timer on the Arduino Nano 33 BLE Sense (nRF52840) was successfully configured, started, fed during normal operation, and — critically — shown to automatically recover the system from a genuine, intentional firmware hang without any human intervention. The reset cause was independently confirmed via the RESETREAS hardware register, providing verifiable proof (not just an assumption) that the recovery was triggered by the watchdog specifically.

8. Conclusion

This case study confirms that the Arduino Nano 33 BLE Sense's onboard hardware Watchdog Timer functions correctly and can be relied upon as an automatic fault-recovery mechanism for embedded and IoT firmware. Because the recovery happens at the silicon level, it works even when the CPU is completely frozen — a condition no purely software-based safeguard could recover from on its own. This isolated result de-risks and validates the same WDT configuration approach used in the larger Reliable IoT Weather Station firmware, where this exact mechanism (see WatchdogController.cpp and ResetDiagnostics.cpp) is combined with sensor reading, fault detection, and a recovery state machine to build a self-healing weather station.